CS 350 Task 4: Project Proof-of-Concept Component Development

Description

This task takes the presumed negotiated final form of Task 3 in combination with the results of Task 2 and our understanding from Task 1 to define a critical component that we want to investigate in a simple form. In this case, it serves to move a value across a numerical interval. It could map to almost any element from Task 2 that moves, but we are concerned only with the interval behavior. Lecture 21 explains the behavior.

There are three parts:

Pretask:	A free-form request to me for clarification on these specifications. This must be plain text.
Actual Task:	Defined below.
Posttask:	A reflection on your solution and its development process after receiving graded feedback on it. The format of this part will be provided later.

Requirements

You are writing a simple Java solution for creating arbitrary components of two types. Both are a form of counter that maintains an arbitrary double value as a state on an interval. The state is initialized to the start of the interval and advances toward the end by a step once per update. The linear component advances by a fixed step. The nonlinear component advances by a step that itself changes once per update, which models simple acceleration.

An appropriate solution requires only programming skills that you already have from the previous courses.

Specifications

I. General

a. All code must reside in package `s19cs350task4`.
b. Include appropriate `toString()` methods.
c. Include appropriate error checking that throws a `RuntimeException`.
d. Unless otherwise specified, you may implement your solution any way you want.
e. You must document your code with basic Javadoc.
f. Use proper object-oriented programming, and especially consider the extensibility of your solution.
g. All methods are public.

II. Create an interface called `I_Component` with the following contract:

`String getID_()`
Gets the arbitrary nonempty identifier of this component.

`double getState_()`
Gets the current state, which is always between the start and end states.

`double getStateStart_()`
Gets the initial state that this state assumes.

`double getStateEnd_()`
Gets the final state that this state can assume.

`double getStep_()`
Gets the value of the transition step between the start and end states. It is always positive.

`boolean updateState_()`
Updates the state from its current state to its next state based on the current step. If the next state exceeds the end state, then the former is clamped to the latter. This returns whether the end state has been reached.

`void cancel_()`
Immediately stops the component from servicing calls to `updateState_()`.

void terminate_()
Stops the component from servicing calls to `updateState_()` with a notional gradual shutdown. For linear components, reverse the step and cease servicing three calls after receipt of this terminate signal. For nonlinear components, also reduce the step by half each time.

boolean isDying_()
Returns whether the component is being terminated.

boolean isDead_()
Returns whether the component has been canceled or terminated.

III. Create a concrete public class called `MyComponentLinear` that implements `I_Component` such that calling `updateState_()` increments or decrements the current state by the step once per call until the end state is reached.

The constructor is:

```
MyComponentLinear(String id, double stateStart, double stateEnd, double step)
```

IV. Create a concrete public class called `MyComponentNonlinear` that implements `I_Component` such that calling `updateState_()` increments or decrements the current state by the current step once per call until the end state is reached, and the step value then increments by **`stepAcceleration`**.

The constructor is:

```
MyComponentNonlinear(String id, double stateStart, double stateEnd, double step,
                     double stepAcceleration)
```